GIT RE-BASIN: MERGING MODELS MODULO PERMUTATION SYMMETRIES

Samuel K. Ainsworth, Jonathan Hayase, Siddhartha Srinivasa
Paul G. Allen School of Computer Science and Engineering
University of Washington
{skainswo,jhayase,siddh}@cs.washington.edu

arXiv:2209.04836v6 [cs.LG] 1 Mar 2023

ABSTRACT

The success of deep learning is due in large part to our ability to solve certain massive non-convex optimization problems with relative ease. Though non-convex optimization is NP-hard, simple algorithms – often variants of stochastic gradient descent – exhibit surprising effectiveness in fitting large neural networks in practice. We argue that neural network loss landscapes often contain (nearly) a single basin after accounting for all possible permutation symmetries of hidden units a la Entezari et al. (2021). We introduce three algorithms to permute the units of one model to bring them into alignment with a reference model in order to merge the two models in weight space. This transformation produces a functionally equivalent set of weights that lie in an approximately convex basin near the reference model. Experimentally, we demonstrate the single basin phenomenon across a variety of model architectures and datasets, including the first (to our knowledge) demonstration of zero-barrier linear mode connectivity between independently trained ResNet models on CIFAR-10. Additionally, we investigate intriguing phenomena relating model width and training time to mode connectivity. Finally, we discuss shortcomings of the linear mode connectivity hypothesis, including a counterexample to the single basin theory.

1 INTRODUCTION

We investigate the unreasonable effectiveness of stochastic gradient descent (SGD) algorithms on the high-dimensional non-convex optimization problems of deep learning. In particular,

1. Why does SGD thrive in optimizing high-dimensional non-convex deep learning loss landscapes despite being noticeably less robust in other non-convex optimization settings, like policy learning (Ainsworth et al., 2021), trajectory optimization (Kelly, 2017), and recommender systems (Kang et al., 2016)?
2. What are all the local minima? When linearly interpolating between initialization and final trained weights, why does the loss smoothly and monotonically decrease (Goodfellow & Vinyals, 2015; Frankle, 2020; Lucas et al., 2021; Vlaar & Frankle, 2021)?
3. How can two independently trained models with different random initializations and data batch orders inevitably achieve nearly identical performance? Furthermore, why do their training loss curves often look identical?

We posit that these phenomena point to the existence of some yet uncharacterized invariance(s) in the training dynamics causing independent training runs to exhibit similar characteristics. Hecht-Nielsen (1990) noted the permutation symmetries of hidden units in neural networks; briefly, one can swap any two units of a hidden layer in a network and – assuming weights are adjusted accordingly – network functionality will not change. Recently, Benton et al. (2021) demonstrated that SGD solutions form a connected volume of low loss and Entezari et al. (2021) conjectured that this volume is convex modulo permutation symmetries.

Conjecture 1 (Permutation invariance, informal (Entezari et al., 2021))**.** Most SGD solutions belong to a set whose elements can be permuted so that no barrier (as in Definition 2.2) exists on the linear interpolation between any two permuted elements.

ARCHITECTURE	NUM. PERMUTATION SYMMETRIES
MLP (3 layers, 512 width)	$10 \wedge 3498$
VGG16	$10 \wedge 35160$
ResNet50	$10 \wedge 55109$
Atoms in the observable universe	$10 \wedge 82$

Table 1: **Permutation symmetries of deep learning models vs. an upper estimate on the number of atoms in the known, observable universe.** Deep learning loss landscapes contain incomprehensible amounts of geometric repetition.

We refer to such solutions as being *linearly mode connected* (LMC) (Frankle et al., 2020), an extension of mode connectivity (Garipov et al., 2018; Draxler et al., 2018). If true, Conjecture 1 will both materially expand our understanding of how SGD works in the context of deep learning and offer a credible explanation for the preceding phenomena, in particular.

Contributions. In this paper, we attempt to uncover what invariances may be responsible for the phenomena cited above and the unreasonable effectiveness of SGD in deep learning. We make the following contributions:

1. **Matching methods.** We propose three algorithms, grounded in concepts and techniques from combinatorial optimization, to align the weights of two independently trained models. Where appropriate, we prove hardness results for these problems and propose approximation algorithms. Our fastest method identifies permutations in mere seconds on current hardware.
2. **Relationship to optimization algorithms.** We demonstrate by means of counterexample that linear mode connectivity is an emergent property of training procedures, not of model architectures. We connect this result to prior work on the implicit biases of SGD.
3. **Experiments, including zero-barrier LMC for ResNets.** Empirically, we explore the existence of linear mode connectivity modulo permutation symmetries in experiments across MLPs, CNNs, and ResNets trained on MNIST, CIFAR-10, and CIFAR-100. We contribute the first-ever demonstration of zero-barrier LMC between two independently trained ResNets. We explore the relationship between LMC and model width as well as training time. Finally, we show evidence of our methods' ability to combine models trained on independent datasets into a merged model that outperforms both input models in terms of test loss (but not accuracy) and is no more expensive in compute or memory than either input model.

2 Background

Although our methods can be applied to arbitrary model architectures, we proceed with the multilayer perceptron (MLP) for its ease of presentation (Bishop, 2007). Consider an L-layer MLP,

$$f(\boldsymbol{x}; \Theta) = \boldsymbol{z}_{L+1}, \quad \boldsymbol{z}_{\ell+1} = \sigma(\boldsymbol{W}_\ell \boldsymbol{z}_\ell + \boldsymbol{b}_\ell), \quad \boldsymbol{z}_1 = \boldsymbol{x},$$

where σ denotes an element-wise nonlinear activation function. Furthermore, consider a loss, $\mathcal{L}(\Theta)$, that measures the suitability of a particular set of weights Θ towards some goal, e.g., fitting to a training dataset.

Central to our investigation is the phenomenon of *permutation symmetries* of weight space. Given Θ, we can apply some permutation to the output features of any intermediate layer, ℓ, of the model, denoted by a permutation matrix $\boldsymbol{P} \in S_d$,[1]

$$\boldsymbol{z}_{\ell+1} = \boldsymbol{P}^\top \boldsymbol{P} \boldsymbol{z}_{\ell+1} = \boldsymbol{P}^\top \boldsymbol{P} \sigma(\boldsymbol{W}_\ell \boldsymbol{z}_\ell + \boldsymbol{b}_\ell) = \boldsymbol{P}^\top \sigma(\boldsymbol{P} \boldsymbol{W}_\ell \boldsymbol{z}_\ell + \boldsymbol{P} \boldsymbol{b}_\ell)$$

for σ, an element-wise operator. It follows that as long as we reorder the input weights of layer $\ell + 1$ according to $\boldsymbol{P}^\top$, we will have a functionally equivalent model. To be precise, if we define Θ' to be identical to Θ with the exception of

$$\boldsymbol{W}'_\ell = \boldsymbol{P} \boldsymbol{W}_\ell, \quad \boldsymbol{b}'_\ell = \boldsymbol{P} \boldsymbol{b}_\ell, \quad \boldsymbol{W}'_{\ell+1} = \boldsymbol{W}_{\ell+1} \boldsymbol{P}^\top,$$

[1] We denote the set of all $d \times d$ permutation matrices – isomorphic to the symmetric group – as S_d, to the possible chagrin of pure mathematicians.

then the two models are functionally equivalent: $f(\boldsymbol{x};\Theta) = f(\boldsymbol{x};\Theta')$ for all inputs $\boldsymbol{x}$. This implies that for any trained weights Θ, there is an entire equivalence class of functionally equivalent weight assignments, not just one such Θ, and convergence to any one specific element of this equivalence class, as opposed to any others, is determined only by random seed. We denote a functionality-preserving permutation of weights as $\pi(\Theta)$.

Consider the task of reconciling the weights of two, independently trained models, A and B, with weights Θ_A and Θ_B, respectively, such that we can linearly interpolate between them. We assume that models A and B were trained with equivalent architectures but different random initializations, data orders, and potentially different hyperparameters or datasets, as well. Our central question is: Given Θ_A and Θ_B, can we identify some π such that when linearly interpolating between Θ_A and $\pi(\Theta_B)$, all intermediate models enjoy performance similar to Θ_A and Θ_B?

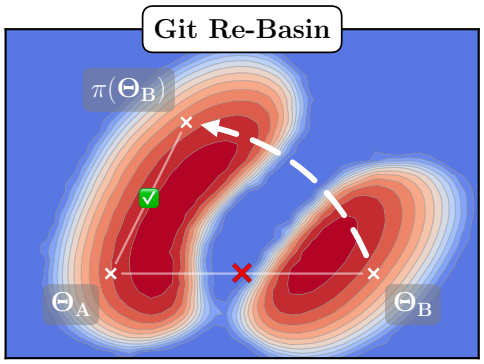

Figure 1: **Git Re-Basin merges models by teleporting solutions into a single basin.** Θ_B is permuted into functionally-equivalent $\pi(\Theta_B)$ so that it lies in the same basin as Θ_A.

We base any claims of loss landscape convexity on the usual definition of multi-dimensional convexity in terms of one-dimensional convexity per

Definition 2.1 (Convexity)**.** A function $f : \mathbb{R}^D \to \mathbb{R}$ is convex if every one-dimensional slice is convex, i.e., for all $x, y \in \mathbb{R}^D$, the function $g(\lambda) = f((1-\lambda)x + \lambda y)$ is convex in λ.

Due to Definition 2.1, it suffices to show that arbitrary one-dimensional slices of a function are convex in order to reason about the convexity of complex, high-dimensional functions. In practice, we rarely observe perfect convexity but instead hope to approximate it as closely as possible. Following Frankle et al. (2020); Entezari et al. (2021); Draxler et al. (2018); Garipov et al. (2018) and others, we measure approximations to convexity via "barriers."

Definition 2.2 (Loss barrier (Frankle et al., 2020))**.** Given two points Θ_A, Θ_B such that $\mathcal{L}(\Theta_A) \approx \mathcal{L}(\Theta_B)$, the *loss barrier* is defined as $\max_{\lambda\in[0,1]} \mathcal{L}((1-\lambda)\Theta_A + \lambda\Theta_B) - \frac{1}{2}(\mathcal{L}(\Theta_A) + \mathcal{L}(\Theta_B))$.

Loss barriers are non-negative, with zero indicating an interpolation of flat or positive curvature.

3 Permutation Selection Methods

We introduce three methods of matching units between model A and model B. Further, we present an extension to simultaneously merging multiple models in Appendix A.10 and an appealing but failed method in Appendix A.11.

3.1 Matching Activations

Following the classic Hebbian mantra, "[neural network units] that fire together, wire together" (Hebb, 2005), we consider associating units across two models by performing regression between their activations. Matching activations between models is compelling since it captures the intuitive notion that two models must learn similar features to accomplish the same task (Li et al., 2016). Provided activations for each model, we aim to associate each unit in A with a unit in B. It stands to reason that a linear relationship may exist between the activations of the two models. We fit this into the regression framework by constraining ordinary least squares (OLS) to solutions in the set of permutation matrices, S_d. For activations of the ℓ'th layer, let $\boldsymbol{Z}^{(A)}, \boldsymbol{Z}^{(B)} \in \mathbb{R}^{d\times n}$ denote the d-dim. activations for all n training data points in models A and B, respectively. Then,

$$\boldsymbol{P}_\ell = \underset{\boldsymbol{P}\in S_d}{\arg\min} \sum_{i=1}^{n} \|\boldsymbol{Z}^{(A)}_{:,i} - \boldsymbol{P}\boldsymbol{Z}^{(B)}_{:,i}\|^2 = \underset{\boldsymbol{P}\in S_d}{\arg\max} \langle \boldsymbol{P}, \boldsymbol{Z}^{(A)}(\boldsymbol{Z}^{(B)})^\top \rangle_F, \tag{1}$$